# MESH: Efficient and Accurate Multi-Hop Retrieval of Confidential Enterprise Knowledge in Trusted Execution Environments

Fan Yusen
National University of Singapore
yusenfan@u.nus.edu

Bharadwaj Veeravalli
National University of Singapore
elebv@nus.edu.sg

Bharath Ramesh
CEO, FlintxAI
bharathr@flintxai.com

***Abstract*—Enterprise question answering often requires connecting evidence that is distributed across multiple confidential documents. Protecting this knowledge during processing restricts the memory and latency budget available for retrieval and inference. We present MESH (Multi-hop Evidence Search over Hierarchies), a wiki-based retrieval framework for confidential multi-hop question answering. Offline, MESH compiles source documents into linked knowledge pages that retain source references and record how new facts relate to earlier ones. It then summarizes overlapping groups of directly linked pages into navigation summaries, which are indexed with hybrid BM25 and dense retrieval. Online, an agent tracks unresolved evidence requirements and uses them to steer bounded iterative search. Each answer claim must resolve to text the agent actually read, and summaries are used only for navigation. We also propose a deployment design in which retrieval runs inside an Intel TDX confidential virtual machine and inference runs on an NVIDIA H100 in confidential-computing mode, with storage keys released only after attestation. On subsets of HotpotQA and 2WikiMultiHopQA, MESH achieves the highest F1 on HotpotQA (0.798) and the highest supporting-page recall on 2WikiMultiHopQA (0.950). It remains competitive with LLM-Wiki in answer accuracy while reducing average answering time by 31–35% and retrieval steps by 14–30%.**

## I. Introduction

Enterprises maintain sensitive knowledge across documents such as internal reports, operational procedures, technical specifications, and contractual records [1]. Answering questions over these collections requires identifying relevant documents, retrieving supporting evidence, and synthesizing that evidence into a correct answer. This process becomes more challenging for multi-hop questions, where the necessary information is distributed across multiple documents and cannot be obtained from a single passage [2]. For example, determining whether a proposed operational change complies with an internal policy may require connecting requirements in one document with technical details and exceptions recorded elsewhere. The resulting research problem is how to retrieve complete cross-document evidence and generate accurate answers while protecting the underlying enterprise documents from unauthorized disclosure during processing.

Protecting enterprise knowledge requires addressing external attacks that exploit vulnerable applications, stolen credentials, or weaknesses in shared cloud infrastructure to gain unauthorized access to sensitive data [3]. The established safeguards include authentication and access control, network boundary protection, security monitoring, and encryption of data at rest and in transit [4]. However, storage and transport encryption alone do not protect plaintext once it is decrypted for conventional processing. If an attacker compromises the operating system or hypervisor, sensitive application memory may remain exposed despite these safeguards. In document retrieval and answer generation, this risk extends to retrieved passages and intermediate representations used during computation. This gap motivates the use of trusted execution environments to complement existing controls with hardware-supported isolation of sensitive computation from potentially untrusted hosting components [5].

Trusted execution environments (TEEs) provide a foundation for protecting sensitive knowledge during computation [6], among other privacy enhancing technologies [7]. TEEs address the confidentiality of data while it is being processed. Depending on the platform and its threat model, a TEE isolates application code and data from potentially untrusted components of the hosting environment, including privileged system software. Remote attestation can additionally provide evidence about the identity and configuration of the execution environment before sensitive data is released [5]. These capabilities make TEEs a suitable foundation for confidential enterprise retrieval. However, maintaining confidentiality requires protecting both source documents and their derived representations throughout the pipeline, from document ingestion and embedding generation to indexing, retrieval, and answer generation.

Large language models (LLMs) can support enterprise question answering by interpreting queries and synthesizing retrieved evidence into natural-language answers [8]. Confidential virtual machines (CVMs), together with confidential-computing-capable GPUs, make it possible to run LLM inference within a protected CPU–GPU environment, as demonstrated by evaluations of modern CPU TEEs and NVIDIA H100 confidential GPUs [9]. However, the lifetime of an enterprise knowledge collection typically extends beyond that of individual compute instances. In deployments using short-lived CVMs, source documents, embeddings, and indexes can therefore be persisted in encrypted external storage and loaded into the trusted environment when needed. Following

an attestation-gated key-release design, decryption keys can be provisioned only after the execution environment satisfies the owner's verification policy [10]. Embeddings require protection alongside source documents: they are not encrypted representations, and embedding-inversion attacks can recover sensitive information from them [11]. Resource constraints also remain relevant, since model weights and inference key–value caches consume substantial memory, while longer input sequences increase cache requirements [12]. These considerations motivate a retrieval architecture that maintains persistent encrypted knowledge outside individual compute instances while performing evidence processing and answer generation within the trusted boundary.

Confidential execution does not, by itself, resolve the difficulties of accurate and efficient multi-hop retrieval. Locating individually relevant passages is insufficient when an answer depends on several connected pieces of evidence. Retrieving too little information may omit a necessary reasoning step, whereas retrieving a large volume of loosely related content increases processing cost and can obscure the evidence needed for the answer. Iterative retrieval can address missing information, but additional model calls and retrieval rounds also increase end-to-end latency. When retrieval and inference are distributed across CPU and confidential GPU environments, the system must also maintain an appropriate trust boundary across those components and protect data exchaneg between them [10]. The challenge therefore concerns both the organization of knowledge and the coordination of retrieval within a confidential processing pipeline.

Existing approaches address different aspects of this problem. Retrieval-augmented generation (RAG) combines external knowledge retrieval with language-model generation, providing a framework for answering questions using document collections [8]. Approximate nearest neighbor (ANN) search accelerates the retrieval of nearby vectors in an embedding space [13], while lexical methods such as BM25 [14] provide complementary signals based on term matching. However, efficient similarity search does not necessarily recover all evidence required for a multi-hop answer. Wiki-style knowledge organization offers another approach: Andrej Karpathy's LLM Wiki proposal describes how an LLM can transform source documents into a persistent collection of interlinked Markdown pages, including entity pages, concept pages, and synthesized summaries [15]. This structure organizes knowledge across documents and preserves connections that can be reused when answering subsequent questions. These approaches motivate the investigation of a wiki-style structure that makes cross-document relationships explicit while retaining access to the underlying evidence. In this work, we examine how such a structure can support multi-hop retrieval within a TEE-based system.

We propose a confidential enterprise retrieval system centered on a hierarchical, wiki-style representation of document knowledge. The system combines document embeddings with BM25-based lexical retrieval, allowing queries to draw on both semantic similarity and explicit terminology. Enterprise documents are organized into linked knowledge pages and higher-level summaries, while source references connect this derived representation to the original evidence. A retrieval agent uses the structure to identify candidate pages, decompose multi-hop questions, and retrieve additional supporting material before answer generation. Open-source language models operate within an attested confidential GPU environment, and the proposed architecture keeps sensitive document processing and inference within the defined trusted boundary. The hierarchy is intended to guide retrieval toward relevant evidence, while source-level access limits reliance on potentially incomplete summaries.

The central contribution of this work is a TEE-based, wiki-style retrieval mechanism for confidential enterprise multi-hop question answering. It integrates hierarchical knowledge organization, hybrid lexical and semantic retrieval, and agent-guided evidence gathering within a confidential execution architecture. The mechanism targets three objectives: accuracy through the recovery of cross-document supporting evidence; efficiency through focused retrieval and controlled iterative processing; and confidentiality through protection of original documents and sensitive intermediate representations during computation. Its effectiveness must be assessed through evidence-retrieval quality, answer correctness, end-to-end latency, and analysis of the system's stated security boundary.

## II. Related Work

Our work draws on three lines of research: retrieval-augmented generation and structured retrieval, knowledge compilation and multi-hop reasoning, and confidential computing for sensitive retrieval and generation.

### A. Confidential Computing for Retrieval and Generation

Confidential computing addresses exposure of sensitive data during execution by placing computation within a hardware-supported trusted boundary. SGX-IR [16] uses Intel SGX to securely outsource encrypted-index construction to the cloud, reducing the computational burden on data owners. It builds encrypted indexes for TF-IDF-based ranked text retrieval and supports eigenface-based face recognition. Its data-oblivious algorithms reduce access-pattern leakage, while blocking and LRU caching, adopted from SGX-BigMatrix, accommodate limited enclave memory. This work establishes secure index construction as an important part of confidential retrieval, beyond protecting access to an existing index.

Supercharging Federated Intelligence Retrieval [17] extends confidential computing to federated RAG: retrieval occurs within local data silos, while server-side aggregation and text generation execute in an attested confidential computing environment. Its emphasis is on protected cross-silo aggregation and remote inference.

While SGX-IR focuses on secure search-index construction, our work investigates Wiki-based knowledge organization and multi-hop evidence acquisition within a proposed confidential CPU–GPU architecture. The Wiki is compiled from enterprise documents into linked knowledge pages and navigation

summaries. These representations are persisted in encrypted storage and loaded into an Intel TDX confidential VM for decryption, retrieval, and evidence validation. Keeping sensitive data within the protected guest during CPU-side processing limits plaintext exposure to the untrusted host. Model inference runs on an NVIDIA H100 in confidential-computing mode, with protected communication between the CPU and GPU.

### *B. Retrieval-Augmented Generation and Structured Retrieval*

Retrieval-augmented generation (RAG) combines external retrieval with language-model generation. Lewis et al. [18] introduce a framework that integrates a dense passage index with a sequence-to-sequence model, allowing generation conditioned on retrieved knowledge. This provides a foundation for document-grounded question answering, but relevance to the original query alone may not identify intermediate evidence needed for cross-document reasoning.

Lexical retrieval provides a complementary way to identify relevant text. BM25 [14] ranks documents using query-term matches, inverse document frequency, term-frequency saturation, and document-length normalization. This makes it useful for queries containing explicit terminology, such as policy names, technical terms, and entity names. Dense retrieval can capture semantic similarity when queries and evidence use different wording. These complementary signals motivate our use of BM25 and dense retrieval over navigation summaries, with their rankings combined through reciprocal rank fusion.

Structured retrieval methods organize knowledge beyond individual passages. RAPTOR [19] recursively embeds, clusters, and summarizes text to construct a tree containing multiple levels of abstraction. Retrieval over this tree provides access to both detailed passages and broader summaries. GraphRAG [20] extracts entities and relationships, constructs a graph, and generates hierarchical community summaries. Its global search approach uses these summaries to produce and aggregate partial answers to corpus-level questions.

RAPTOR inspires our use of higher-level summaries as compact representations of information distributed across documents, while GraphRAG motivates the use of explicit relationships to connect related knowledge. We adapt these ideas by constructing navigation summaries over overlapping groups of linked Wiki pages.

### *C. Knowledge Compilation and Multi-Hop Reasoning*

Knowledge compilation transforms source documents into a persistent representation that can be reused across questions. Our Wiki-style representation is inspired by Andrej Karpathy's LLM Wiki proposal [15], which describes an LLM-maintained collection of interlinked Markdown pages built from preserved raw sources. As new material arrives, the LLM integrates it into entity and concept pages, updates summaries and cross-references, and records contradictions.

LLM-Wiki [21] develops a concrete agent-native retrieval system that compiles documents into structured Markdown pages with source references and bidirectional links. It exposes search, reading, and link-following operations to an agent, which adapts its traversal to intermediate observations and assesses whether sufficient evidence has been collected. Its Error Book records recurring construction errors and supports structural and semantic repair. This work is the closest antecedent to our combination of offline Wiki construction and online agent-controlled retrieval.

IRCoT [22] interleaves retrieval with chain-of-thought steps: intermediate reasoning guides subsequent retrieval, while retrieved passages inform later reasoning. It demonstrates the importance of adapting evidence acquisition as a multi-step answer develops, rather than relying solely on an initial retrieval.

Building on these approaches, Our method explicitly tracks missing evidence during retrieval and maintains traceable links between answer claims and the text read by the agent. Incremental compilation also retains earlier facts and records additions, temporal updates, corrections, and conflicts. These mechanisms make evidence acquisition and knowledge changes inspectable.

## III. METHOD

### *A. Overview*

We propose MESH (Multi-hop Evidence Search over Hierarchies), a Wiki-based retrieval framework that separates offline knowledge construction from online question answering, as shown in Fig. 1. MESH organizes document knowledge into three layers: source archives preserve the original content, linked knowledge pages consolidate facts with source references, and navigation summaries provide a higher-level view for locating relevant evidence.

During offline construction (Fig. 1(a)), source documents are archived and compiled into knowledge pages that contain facts, source references and explicit links to related pages. Updates preserve earlier knowledge while incorporating new information. MESH then forms overlapping groups of linked pages, generates navigation summaries, and builds a hybrid BM25 and dense index over these summaries.

During online question answering (Fig. 1(b)),the agent identifies the information needed to answer the question and uses summary retrieval to locate candidate knowledge pages. It reads these pages and, when necessary, their original sources, and then uses remaining evidence gaps to guide subsequent searches. The evidence identifiers connect the final answer to the text returned during reading. Summaries guide navigation, while answer evidence comes from the knowledge pages and source passages actually read.

MESH combines relationship-guided navigation, explicit tracking of missing evidence, and bounded iterative retrieval. In the proposed confidential deployment, document processing and information retrieval execute inside an Intel TDX confidential virtual machine, while model inference runs on an NVIDIA H100 in confidential-computing mode. Wiki pages and indexes are persisted in encrypted storage and loaded into the protected environment when needed.

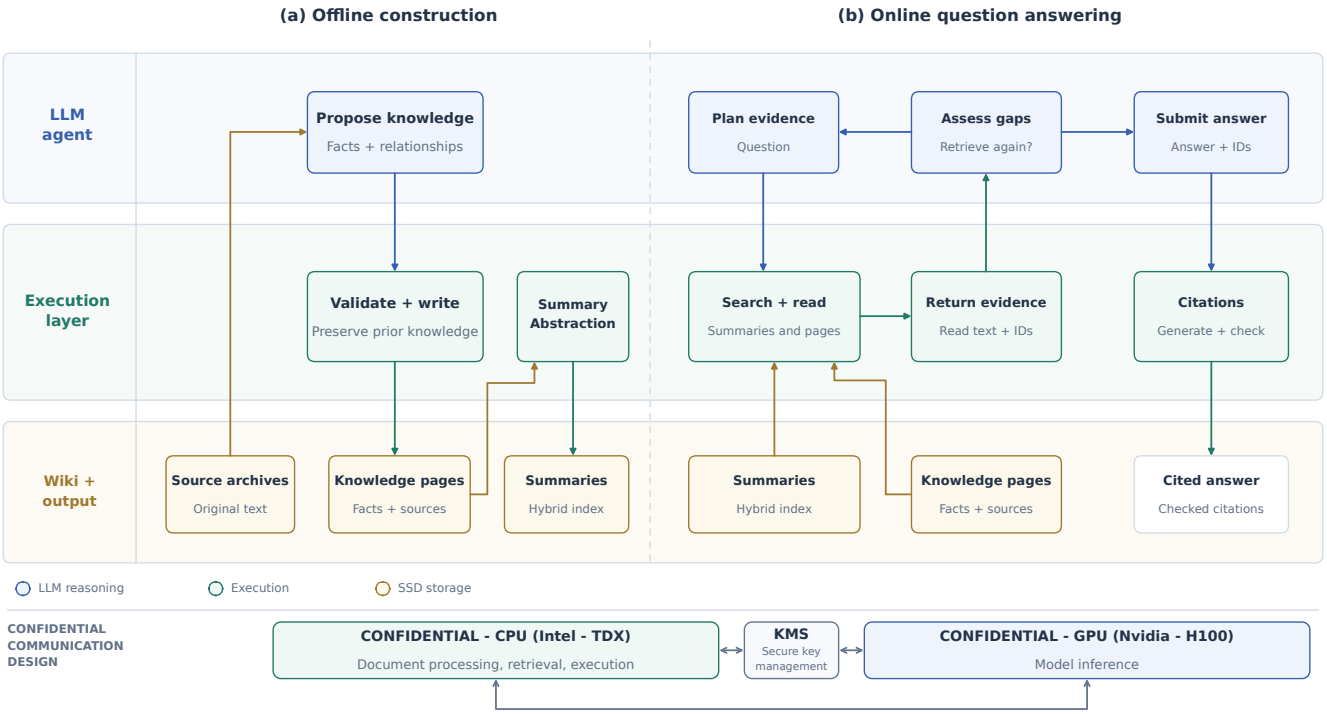



Fig. 1. System overview of MESH. (a) Offline construction transforms archived sources into validated knowledge pages and navigation summaries. (b) Online question answering alternates evidence planning, search and reading, and gap assessment before submitting an answer with evidence identifiers for citation generation and checking. The lower panel shows the proposed confidential CPU–GPU deployment with key management.

### B. Source-Preserving Wiki Compilation

*1) Archiving and page selection:* Facts about the same entity or policy often appear in different documents. We compile them into reusable knowledge pages while retaining their sources. Each processed document is archived under its SHA-256 content hash. Changed content creates a new archive entry.

For each batch of documents, a selection model receives the new documents and the existing page catalog. It selects relevant pages to update or link. The system reads these pages and their cited source documents before generating changes.

*2) Generation and validation:* The LLM receives the document batch, selected pages, existing evidence, and allowed page types. It proposes page metadata, facts, source references, and related-page links.

Before writing knowledge pages, the execution layer validates page paths, required fields, source references, relationship targets, and incremental update records. Each fact must cite at least one supplied source and every input document must be cited in the proposal. Existing pages can be updated only if their contents were provided to the model.

Rejected proposals receive validation feedback for up to two corrective retries. If proposal validation still fails for a multi-document batch, the system retries its documents individually. Accepted proposals are rendered as Markdown, and successful document versions are recorded for reuse. These checks enforce structural consistency and source-reference integrity.

*3) Incremental preservation:* New documents may add information, revise a policy, or conflict with earlier statements. The system preserves earlier facts and appends new ones. Each new fact on an existing page records its relation to prior knowledge: addition, elaboration, temporal update, correction, or conflict.

The record includes a reason, related earlier fact identifiers and a source-supported validity time when available. This lets earlier and newer statements coexist without losing their sources. The resulting linked pages provide the input to summary construction and later evidence reading.

### C. Relationship-Guided Hybrid Navigation

*1) Overlapping page groups:* Supporting evidence may span several knowledge pages. To expose these pages together, we form a group from each page and its explicit Related Pages links:

$$G_p = \{p\} \cup R(p), \tag{1}$$

where $R(p)$ contains the existing knowledge pages directly linked from page $p$. We deduplicate identical groups and remove strict subsets. For the unique candidate family $\mathcal{G}_0$, the retained groups are

$$\mathcal{G} = \{G \in \mathcal{G}_0 : \nexists H \in \mathcal{G}_0 \text{ with } G \subsetneq H\}. \tag{2}$$

Groups can overlap. For example, $\{A, B, C\}$ and $\{A, B, D\}$ both remain, while $\{A, B\}$ is removed. This preserves different contexts around shared pages. Grouping uses direct links rather than recursively merging all connected pages.

The LLM writes a navigation summary for each multi-page group. Singleton summaries are generated directly from their pages. The execution layer fixes membership and adds page links. A summary is used only when its membership and content fingerprint match the loaded Wiki snapshot.

*2) Hybrid retrieval:* Enterprise queries contain both exact terms and paraphrased descriptions. We therefore combine BM25 with dense retrieval over summary text. Long summaries are split into consecutive chunks. For a normalized encoder $e$, the dense score is the highest chunk similarity:

$$D(q, s) = \max_j e(q)^\mathsf{T} e(c_{s,j}), \tag{3}$$

where $q$ is the query and $c_{s,j}$ is a chunk of summary $s$. The two rankings are combined through reciprocal rank fusion:

$$F(q, s) = \sum_{\substack{\ell \in \{\mathrm{BM25,dense}\} \\ s \in L_\ell(q)}} \frac{1}{k_{\mathrm{RRF}} + \mathrm{rank}_\ell(s)}. \tag{4}$$

Returned results are limited by count and each result includes member-page links for the next reading step. Unambiguous title or alias matches can also lead directly to pages. Directory browsing remains available when summaries miss useful content.

The hybrid index contains navigation summaries, not the full source archive. Summary text and retrieval scores guide selection; they do not provide final answer evidence.

### D. Evidence-State Multi-Hop Acquisition

*1) Tracking missing evidence:* A relevant page may support only one part of a multi-hop answer. The QA agent therefore maintains evidence requirements:

$$\mathcal{R}_t = \{(r_j, u_j, z_j, I_j)\}_{j=1}^{m_t}, \tag{5}$$

where $r_j$ is a requirement identifier, $u_j$ describes needed information, $z_j$ is unresolved or supported, and $I_j$ contains selected evidence identifiers. Requirements are updated by identifier, and omitted entries remain in the state.

The agent reads candidate pages and uses unresolved requirements to guide further retrieval. For bridge questions, an intermediate entity found in evidence can become the next query. For comparison questions, the agent gathers evidence for both entities. Knowledge pages can support answers directly; source passages are read when more detail or verification is needed.

*2) Evidence identifiers and citations:* Each actual read registers citable text for the current question. An evidence identifier maps to an exact quotation, content version, and page offsets or source line numbers. Truncated reads register only the delivered text. Summaries, directory listings, and relationship descriptions are navigation material.

The agent submits claims using these identifiers. The execution layer resolves them into citations. A factual answer requires at least one recorded requirement, support for every recorded requirement, and an evidence chain covering all their identifiers. Each claim must refer to a known requirement, and every selected evidence identifier must match text delivered during reading.

Rejected submissions return feedback to the same agent, which gathers additional information by following the existing evidence chain and exploring related materials. This process continues until the agent determines that the evidence is sufficient to provide an answer or that it cannot find the correct answer, in which case it returns unknown.

*3) Controlling retrieval cost:* Repeated search and model calls increase latency. We prepare and cache summary embeddings, pre-read a small page batch, and provide a combined search-and-read operation. This operation returns candidate navigation and selected page evidence together, avoiding an intervening model round. Requirement updates can also accompany answer submission.

Adaptive retrieval scales the number of returned summaries and the token budget with the number of unresolved requirements, up to their configured limits. Later searches exclude summaries already seen, and the combined operation also excludes pages already read. If a search returns no new summaries, the next search retrieves more candidates. Continuation offsets allow further reading of truncated pages.

The controller bounds each question by tool operations, model turns, returned tokens, and optionally elapsed time, charging failed calls and rejected submissions as well. The final operation is reserved for answer submission, and the agent returns unknown if the budget is exhausted first.

### E. Confidential CPU–GPU Execution

*a) Trust boundary:* Fig. 2 shows the proposed deployment. The trusted computing base consists of two hardware-isolated domains: an Intel TDX confidential virtual machine on the CPU side and an NVIDIA H100 operating in CC mode on the GPU side. The host OS, hypervisor, cloud operator, PCIe interconnect, and physical storage are treated as untrusted. All application components run inside the guest. These are Wiki construction; the QA agent, which performs retrieval, evidence tracking, and citation validation; the embedding service; the model runtime with its GPU driver; and a trusted loader. Intermediate artifacts such as summaries, embeddings, prompts, caches, and logs stay in encrypted guest memory and are never exposed to the host in plaintext.

*b) Confidential CPU–GPU path:* LLM inference is offloaded to the H100, whose memory lies outside the TDX protection domain. To extend the trusted boundary to the GPU, the guest driver establishes an SPDM session with the GPU firmware. This session authenticates the device and derives session keys. Data exchanged between guest and GPU is encrypted and staged through bounce buffers in shared memory. The GPU copy engines then decrypt the data into protected device memory. Prompts, retrieved evidence, model

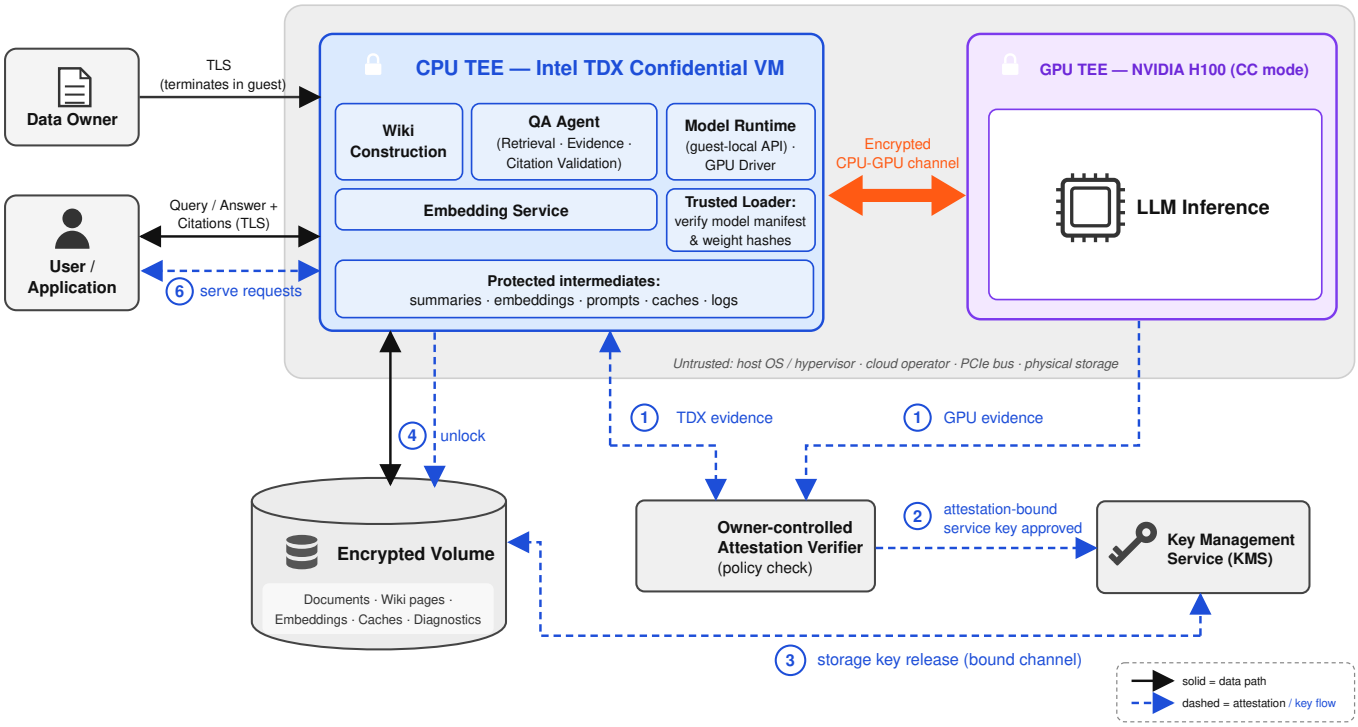



Fig. 2. Proposed confidential deployment design.

inputs, and generated tokens therefore cross the PCIe bus only in ciphertext. The model runtime exposes a guest-local API, so the QA agent never sends requests to an endpoint outside the trusted boundary.

*c) Attestation and key release:* Sensitive processing is gated on a fixed attestation sequence (Fig. 2, steps (1)–(6)). (1) The guest submits fresh TDX evidence and H100 attestation evidence to an owner-controlled verifier. (2) The verifier checks both against the approved policy, covering the guest firmware, kernel, and application measurements, as well as the GPU firmware version, CC-mode status, and device identity. If the evidence satisfies the policy, the verifier approves a service key bound to the attested guest. (3) The key-management service (KMS) releases the storage key only over a channel bound to that service key. (4) The guest unlocks the encrypted volume that holds documents, Wiki pages, embeddings, caches, and diagnostic outputs. Before loading the model onto the GPU, the trusted loader checks the model manifest and weight hashes against approved values. (6) Only then does the service accept requests. User and data-owner TLS connections terminate inside the guest, so queries, answers, and citations are never visible to the host.

*d) Failure handling and scope:* If either CPU or GPU attestation fails, the KMS withholds the storage key. The volume then stays locked and no new sensitive task is admitted. The design's guarantees depend on the approved guest software image, the TDX and H100 CC firmware, and the integrity of the verifier and KMS. This section describes a deployment design. The current retrieval prototype has not established these guarantees through hardware validation.

## IV. Experiment

### A. Experiment Setup

We evaluate the proposed method on two multi-hop question-answering benchmarks, HotpotQA and 2WikiMultiHopQA, to assess its ability to retrieve cross-document evidence and generate accurate answers efficiently. The experiments examine whether relationship-guided navigation and explicit tracking of missing evidence help the agent recover the necessary supporting information while limiting the retrieval effort and latency associated with iterative question answering.

HotpotQA contains crowdsourced questions grounded in Wikipedia articles. Its two main question types are bridge questions, which connect evidence through an intermediate entity, and comparison questions, which require comparing information about different entities. It provides sentence-level supporting facts, enabling evaluation of both answer correctness and supporting evidence. These characteristics make it suitable for evaluating evidence retrieval and integration across documents.

2WikiMultiHopQA combines unstructured Wikipedia text with structured knowledge from Wikidata. It includes four question types: comparison, inference, compositional, and bridge comparison. A distinguishing feature is its explicit reasoning-path annotations, represented as subject–relation–object triples. Questions are constructed using templates and

logical rules designed to require multiple reasoning steps, providing a complementary benchmark for evaluating cross-document evidence integration.

We use the first 500 questions from the HotpotQA distractor development set and the first 100 questions from the 2WikiMultiHopQA development set, preserving their original order rather than randomly sampling questions. All methods are evaluated on the same question subsets. For each dataset, we construct a shared source corpus from the union of the context paragraphs associated with the selected questions, including both supporting and distractor paragraphs. Sentences within each paragraph are concatenated in their original order, and duplicate paragraphs are removed based on exact title–text pairs. Each question is answered using this shared corpus rather than only its individually supplied context. Our experiments therefore evaluate retrieval over dataset-provided corpora rather than over the full Wikipedia collection. Gold answers and supporting annotations are not provided to the construction or question-answering models.

All evaluated methods use GLM-5.2 as their language model. For Wiki-based methods, it is used for offline knowledge construction and online agent execution. For retrieval-based baselines, it generates answers from retrieved evidence. Wherever dense embeddings are required, we use Qwen3-Embedding-8B with 4,096-dimensional representations.

### B. Baselines

We compare the proposed method with LLM-Wiki, RAG, Dense, and BM25. All methods use the same source corpora and answer-generation model, allowing the comparison to examine differences in knowledge organization and evidence acquisition.

- **LLM-Wiki** It compiles source documents into structured Wiki pages with bidirectional links. During online question answering, an agent searches, reads, and follows links to gather evidence, treating retrieval as an iterative reasoning process. The original method also incorporates an Error Book that records recurring structural and semantic errors and converts them into constraints for subsequent construction and repair. For a controlled comparison, We use GLM-5.2 for Wiki construction and online agent execution, including a maximum of 15 tool operations.
- **Dense** This baseline divides source documents into chunks of size 256 Chunks and queries are encoded using Qwen3-Embedding-8B with 3,072 dimensions. For each question, the retriever selects the top five chunks by embedding similarity, and GLM-5.2 generates an answer using these chunks as context. Retrieval is performed once using the original question, without subsequent retrieval based on intermediate evidence.
- **BM25** This baseline uses lexical retrieval to rank source evidence according to BM25 scores. The top five results are supplied to GLM-5.2 for answer generation. It provides a sparse-retrieval comparison for assessing the effectiveness of exact term matching.

TABLE I
ANSWER QUALITY ON HOTPOTQA AND 2WIKIMHQA. SP RECALL DENOTES SUPPORTING-PAGE RECALL. HIGHER IS BETTER; BEST RESULTS ARE BOLDED.

| Method | F1 ↑ | EM ↑ | SP Recall ↑ |
|---|---|---|---|
| **HotpotQA** | | | |
| BM25 | 0.5290 | 0.4200 | 0.7300 |
| Dense | 0.6690 | 0.5200 | **0.8837** |
| WIKI_LLM | 0.7730 | 0.6200 | 0.8360 |
| MESH | **0.7976** | **0.6400** | 0.8800 |
| **2WikiMHQA** | | | |
| BM25 | 0.3500 | 0.3200 | 0.6900 |
| Dense | 0.3600 | 0.3000 | 0.7775 |
| WIKI_LLM | **0.8200** | **0.7500** | 0.8900 |
| MESH | 0.8140 | 0.7200 | **0.9500** |

### C. Implementation Details

Our method retrieves navigation summaries formed from overlapping groups of linked knowledge pages. BM25 and dense rankings are combined using reciprocal rank fusion (RRF). The agent then reads the corresponding knowledge pages and, when needed, original source passages. Navigation summaries guide retrieval but are not accepted as final answer evidence.

The implementation uses BM25 parameters $k_1 = 1.5$ and $b = 0.75$, with an RRF constant of 60. Its default configuration considers 20 candidates from each retriever and returns up to five summaries per search, subject to token limits and adaptive retrieval. Long summaries are split into consecutive chunks of up to 6,000 tokens for embedding. Dense retrieval uses cosine similarity over normalized vectors.

The default latency configuration pre-reads up to two knowledge pages and allows 15 tool operations per question. Summary responses and knowledge-page reads each have a 4,000-token budget. A cooperative question-level time budget of 120 seconds is applied. Document embeddings are cached, and the prepared summary index is loaded or constructed before question processing.

### D. Evaluation Metrics

We evaluate the methods along two dimensions: answer and evidence quality, and online retrieval efficiency. Together, these dimensions assess whether a method obtains sufficient supporting evidence and produces correct answers with limited retrieval effort.

**Answer and evidence quality:**Table 1 reports Exact Match (EM), token-level F1, and supporting-page recall (SP Recall). Before computing EM and F1, predicted and reference answers are normalized by lowercasing, removing English articles and punctuation, and collapsing whitespace. EM assigns a score of one when the normalized prediction exactly matches the reference answer and zero otherwise. Token-level F1 measures answer-token overlap as the harmonic mean of precision and

TABLE II
RETRIEVAL EFFICIENCY ON HOTPOTQA AND 2WIKIMHQA. ALL METRICS ARE AVERAGES. LOWER IS BETTER; BEST RESULTS ARE BOLDED.

| Method | Retrieval Steps ↓ | Pages Read ↓ | Answering Time (s) ↓ |
|---|---|---|---|
| **HotpotQA** | | | |
| BM25 | **1.00** | 4.99 | 5.75 |
| Dense | **1.00** | 4.95 | **3.23** |
| WIKI_LLM | 5.25 | 2.79 | 10.80 |
| MESH | 3.66 | **2.58** | 7.41 |
| **2WikiMHQA** | | | |
| BM25 | **1.00** | 4.75 | **1.80** |
| Dense | **1.00** | 4.50 | 2.62 |
| WIKI_LLM | 4.47 | 2.81 | 9.23 |
| MESH | 3.86 | **2.76** | 6.00 |

recall, providing partial credit for incomplete but overlapping answers.

SP Recall measures the proportion of gold supporting pages covered by the retrieved evidence. Scores are averaged across questions. This metric complements EM and F1 by measuring evidence coverage independently of final answer correctness. It evaluates coverage at the page level, without assessing sentence-level supporting facts or the correctness of the reasoning path.

**Online retrieval efficiency:** Table 2 reports the average retrieval steps, pages read, and answering time per question. Retrieval Steps measures the number of retrieval operations performed during question answering, capturing the extent of iterative evidence acquisition. Pages Read measures the amount of page-level evidence accessed, indicating how broadly each method reads to assemble its answer. Answering Time measures elapsed time from the start of online question processing to the final response, including retrieval, evidence processing, and language-model inference. Offline Wiki construction and index preparation are excluded.

Higher EM, F1, and SP Recall indicate better answer and evidence quality. Lower retrieval steps, pages read, and answering time indicate lower online cost, but should be interpreted alongside the quality metrics: retrieving less evidence is beneficial only when sufficient support and answer accuracy are maintained.

### *E. Ablation Study: Impact of Summary Navigation*

To evaluate the effectiveness of the summary navigation mechanism, we conduct an ablation study on a subset of 500 questions from the HotpotQA dataset. We compare two configurations: (1) the proposed system with summary navigation enabled, and (2) a baseline configuration without summary navigation.

Both configurations are evaluated using answer quality metrics, including F1 score and Exact Match (EM), as well as retrieval efficiency metrics, including the average number of retrieval steps and pages read per question.

TABLE III
ABLATION STUDY OF SUMMARY NAVIGATION ON 500 HOTPOTQA QUESTIONS

| Metric | Without Summary | With Summary |
|---|---|---|
| F1 Score | 0.7155 | **0.7976** |
| Exact Match (EM) | 0.5700 | **0.6400** |
| Avg. Retrieval Steps | 3.85 | **3.66** |
| Avg. Pages Read | **1.86** | 2.58 |

As shown in Table III, enabling summary navigation improves answer quality, increasing the F1 score from 0.7155 to 0.7976 and the Exact Match (EM) score from 0.5700 to 0.6400, corresponding to relative improvements of 11.47% and 12.28%, respectively.

Furthermore, summary navigation reduces the average number of retrieval steps from 3.85 to 3.66, indicating more efficient navigation toward relevant evidence. However, the average number of pages read increases from 1.86 to 2.58, suggesting that summary-guided navigation encourages broader evidence exploration.

Overall, these findings indicate that summary navigation improves answer accuracy and slightly reduces retrieval steps, at the cost of reading more pages per question.

## V. CONCLUSION

This paper presented MESH, a wiki-based retrieval framework for multi-hop question answering over confidential enterprise knowledge. MESH separates offline knowledge construction from online question answering. During construction, source documents are compiled into validated knowledge pages that preserve source references and record additions, temporal updates, corrections, and conflicts. Overlapping groups of directly linked pages are then summarized into navigation summaries and indexed with hybrid BM25 and dense retrieval. During question answering, an agent maintains explicit evidence requirements and uses unresolved requirements to guide bounded iterative retrieval. Answers are accepted only when every claim resolves to evidence that was actually read. We also described a confidential CPU–GPU deployment design that places retrieval, evidence processing, and inference inside an attested Intel TDX and NVIDIA H100 trust boundary, with storage keys released only after verification.

Experiments on HotpotQA and 2WikiMultiHopQA show that relationship-guided navigation and explicit evidence tracking improve the trade-off between answer quality and retrieval cost. On HotpotQA, MESH achieves the highest F1 and EM among the evaluated methods. On 2WikiMultiHopQA, it achieves the highest supporting-page recall, with answer accuracy close to that of LLM-Wiki. Compared with LLM-Wiki, MESH reduces average answering time by 31.4% on HotpotQA and 35.0% on 2WikiMultiHopQA, and requires fewer retrieval steps and fewer pages read on both benchmarks. Single-pass BM25 and dense retrieval answer faster, but their

accuracy is substantially lower, particularly on 2WikiMultiHopQA. These results indicate that a structured wiki representation can support efficient iterative evidence acquisition, which matters in confidential environments where memory and latency budgets are limited.

Several limitations remain. The evaluation of MESH covers only two benchmarks and a limited number of questions, namely 500 from HotpotQA and 100 from 2WikiMultiHopQA, retrieved over dataset-provided corpora rather than full Wikipedia or real enterprise collections. The results therefore need to be confirmed on more datasets and at larger scale. In addition, the confidential deployment is a design and has not yet been validated on TDX and H100 hardware, so the overhead of attestation, encrypted CPU–GPU transfer, and encrypted storage is not reflected in the reported latencies. The evaluation also excludes the cost of offline wiki construction. Future work will evaluate MESH on additional multi-hop benchmarks and on enterprise-style corpora with frequent document updates, measure end-to-end performance on confidential hardware, and quantify construction and incremental maintenance costs.

## REFERENCES


[1] U. Kruschwitz and C. Hull, "Searching the enterprise," *Foundations and Trends in Information Retrieval*, vol. 11, no. 1, pp. 1–142, 2017.

[2] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. W. Cohen, R. Salakhutdinov, and C. D. Manning, "HotpotQA: A dataset for diverse, explainable multi-hop question answering," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2018, pp. 2369–2380. [Online]. Available: https://aclanthology.org/D18-1259/

[3] W. Jansen and T. Grance, "Guidelines on security and privacy in public cloud computing," National Institute of Standards and Technology, Tech. Rep. NIST SP 800-144, 2011.

[4] Joint Task Force, "Security and privacy controls for information systems and organizations," National Institute of Standards and Technology, Tech. Rep. NIST SP 800-53, Revision 5, 2020.

[5] X. Li, B. Zhao, G. Yang, T. Xiang, J. Weng, and R. H. Deng, "A survey of secure computation using trusted execution environments," arXiv:2302.12150, 2023. [Online]. Available: https://arxiv.org/abs/2302.12150

[6] M. Sabt, M. Achemlal, and A. Bouabdallah, "Trusted execution environment: What it is, and what it is not," in *2015 IEEE Trustcom/BigDataSE/ISPA*, vol. 1. IEEE, 2015, pp. 57–64.

[7] E. Ustundag Soykan, L. Karaçay, F. Karakoç, and E. Tomur, "A survey and guideline on privacy enhancing technologies for collaborative machine learning," *IEEE Access*, vol. 10, pp. 97 495–97 519, 2022.

[8] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," arXiv:2312.10997, 2024, first submitted December 2023; revised March 2024. [Online]. Available: https://arxiv.org/abs/2312.10997v5

[9] M. Chrapek, M. Copik, E. Mettaz, and T. Hoefler, "Confidential LLM inference: Performance and cost across CPU and GPU TEEs," 2025. [Online]. Available: https://arxiv.org/abs/2509.18886

[10] NVIDIA, "Deploying proprietary models securely with NVIDIA confidential computing on self-hosted virtual machines," NVIDIA Documentation, 2026, accessed: October 14, 2026. [Online]. Available: https://docs.nvidia.com/enterprise-reference-architectures/deploying-proprietary-models-confidential-compute-self-hosted-vms/latest/index.html

[11] J. X. Morris, V. Kuleshov, V. Shmatikov, and A. M. Rush, "Text embeddings reveal (almost) as much as text," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2023, pp. 12 448–12 460.

[12] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*. ACM, 2023.

[13] A. Andoni, P. Indyk, and I. Razenshteyn, "Approximate nearest neighbor search in high dimensions," arXiv:1806.09823, 2018. [Online]. Available: https://arxiv.org/abs/1806.09823

[14] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009. [Online]. Available: https://doi.org/10.1561/1500000019

[15] A. Karpathy, "LLM Wiki: A pattern for building personal knowledge bases using LLMs," GitHub Gist, Apr. 2026, accessed: October 11, 2026. [Online]. Available: https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f

[16] F. Shaon and M. Kantarcioglu, "SGX-IR: Secure information retrieval with trusted processors," in *Data and Applications Security and Privacy XXXIV*. Springer, 2020, pp. 367–387, 34th Annual IFIP WG 11.3 Conference, DBSec 2020. [Online]. Available: https://www.shaon.dev/assets/files/research/sgx-ir-paper.pdf

[17] D. Stripelis, P. Foley, M. Naseri, W. Lindskog-Münzing, C. S. Ng, D. J. Beutel, and N. D. Lane, "Supercharging federated intelligence retrieval," *arXiv preprint arXiv:2603.25374*, 2026, presented at the Amazon Trusted AI Symposium 2026. [Online]. Available: https://arxiv.org/abs/2603.25374

[18] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474. [Online]. Available: https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html

[19] P. Sarthi, S. Abdullah, A. Tuli, S. Khanna, A. Goldie, and C. D. Manning, "RAPTOR: Recursive abstractive processing for tree-organized retrieval," in *International Conference on Learning Representations*, 2024. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/hash/8a2acd174940dbca361a6398a4f9df91-Abstract-Conference.html

[20] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, D. Metropolitansky, R. O. Ness, and J. Larson, "From local to global: A Graph RAG approach to query-focused summarization," *arXiv preprint arXiv:2404.16130*, 2024. [Online]. Available: https://arxiv.org/abs/2404.16130

[21] H. Ming, F. Li, X. Wu, and W. Que, "Retrieval as reasoning: Self-evolving agent-native retrieval via LLM-Wiki," *arXiv preprint arXiv:2605.25480*, 2026. [Online]. Available: https://arxiv.org/abs/2605.25480

[22] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, "Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2023, pp. 10 014–10 037. [Online]. Available: https://aclanthology.org/2023.acl-long.557/